Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 6
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest



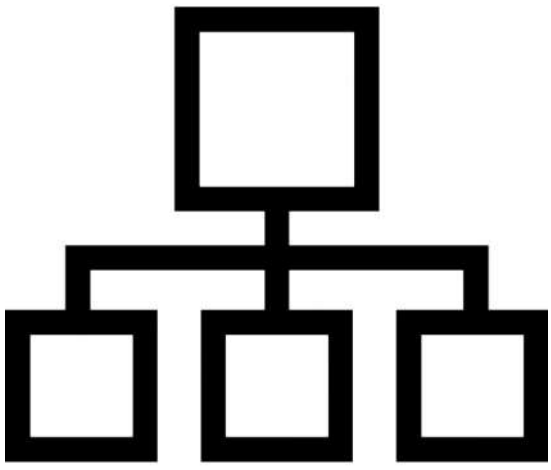
In cursul anterior

- Structura si Setul de Instructiuni SIMD
- Implementarea Salturilor Conditionate in Structuri SIMD
- Organizarea Datelor in Structuri SIMD
- Probleme de Comunicatii intr-un Sistem SIMD
- Deplasarea Ciclica a Datelor intre Procesoare
- Intercalarea Perfecta – Shuffle
- Conectarea Inversa – Inverse Shuffle
- Permutari Elementare



Cuprins

- Comutatoare Ierarhice
- Comutatoare Neierarhice
- Exemple de Comutatoare Procesor Memorie
- Interconectarea in Sistemele cu Resurse Multiple
- Retele de Comutare Ierarhice
- Retele de Comutare de tip Delta
- Retele Bazate pe Rutare
- Performantele Retelelor de Comutare



Comutatoare Ierarhice

Comutatoare

- Se impart in doua mari categorii
  - Ierarhice = conecteaza o componenta de tip a cu una de tip b
    - $a_i \leftrightarrow b_k$ (P ↔ M)



  - Neierarhice = conecteaza doua componenta de acelasi tip a
    - $a_i \leftrightarrow a_k$ (P ↔ P)

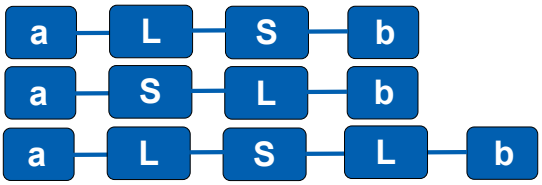


Comutatoare Ierarhice

- Comutatorul Simplu - Poarta
  - Asigura comunicarea intre componente de tip a si b
  - Este descris ca : S {poarta, 1a, 1b}



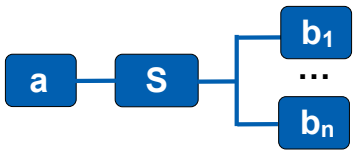
  - Variante posibile:
    - S {poarta, 1a, 1b, comutare la b}
    - S {poarta, 1a, 1b, comutare la a}
    - S {poarta, 1a, 1b, cu legatura la a si b}



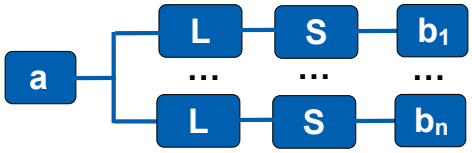
  - Poate conecta o resursa la magistrala sau o magistrala la alta magistrala

Comutatoare Ierarhice Duplex

- Asigura comunicatia intre o componenta de tip a si mai multe componente de tip b
  - Este descris ca: S {poarta, 1a, nb, concurenta 1, 1Sp}



  - Se pot considera toate variantele de la comutatoarele simple



Comutatoare Ierarhice Dual Duplex

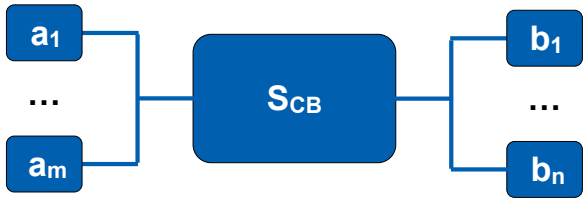
- Asigura comunicatia intre doua componente de tip a si mai multe componente de tip b
  - Este descris ca: S {poarta, 2a, nb, concurenta 2, 2nSp}



  - Sunt posibile aceleasi variante ca la comutatoarele simple

Comutatoare Ierarhice Legatura Multipla

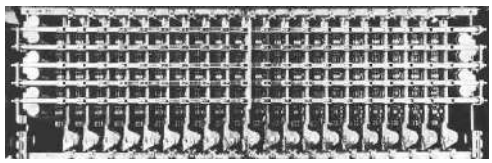
- Asigura comunicatia intre m componente de tip a si n componente de tip b
- Este cunoscut si ca Cross-Bar-Switch
  - S {CB, ma, nb, c = min(m, n), m*nSp}



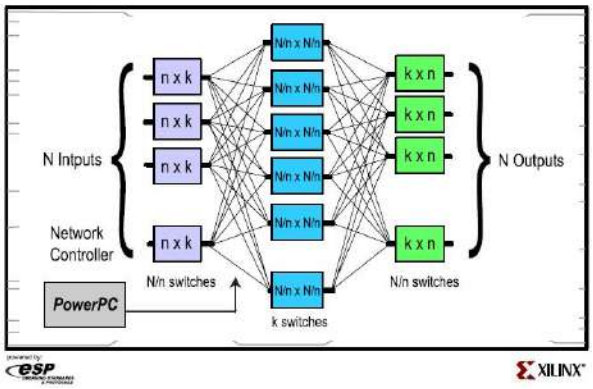
- Orice a poate comunica cu orice b
- Pot avea loc transferuri multiple si simultane!
- Cel mai dificil este de implementat U_{Cmd} pentru S_{CB}
  - Trebuie sa rezolve toate posibilele conflicte

Comutatoare Ierarhice CrossBar Switch

- Cross bar de la Bell System din 1960 cu 20 de verticale si 10 niveluri



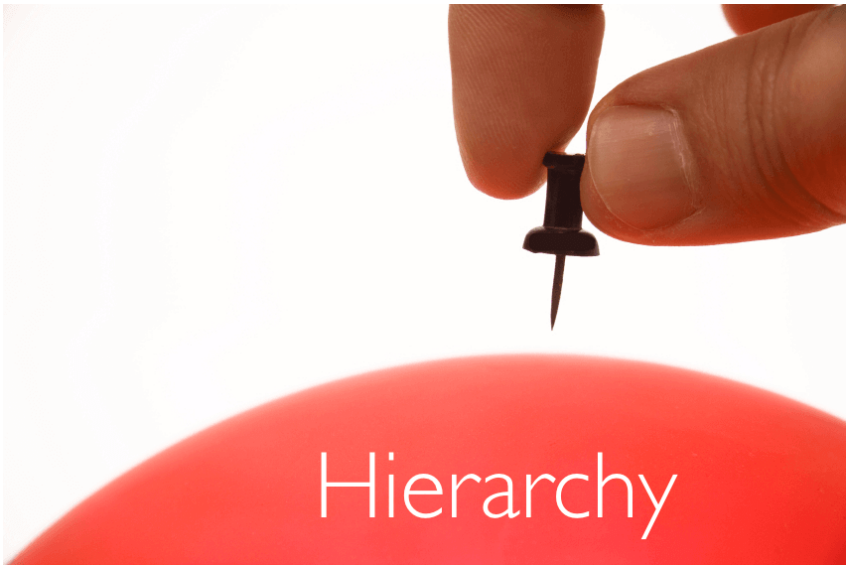
- Un crossbar de la Xilinx in FPGA-urile Virtex-II / Virtex-II Pro/ProX



Comutatoare Ierarhice Trunchi K

- Imbina conectivitatea oferita de S legatura multipla cu costul redus al celorlalte tipuri de comutatoare
  - S {TK, ma, nb, c = k, k*(m+n)Sp}
- Orice a poate comunica cu orice b
- Nu totdeauna vei avea min(m, n) comunicatii simultane
  - Asa a aparut idea de a folosi k linii pentru comunicatie
- In functie de m si n si de trafic, se va determina un k optim
  - 8 procs si 4 memorii:
    - Pot fi maxim 4 accese la memorii → k=4
    - Nu are sens mai mult → k < n si k < m





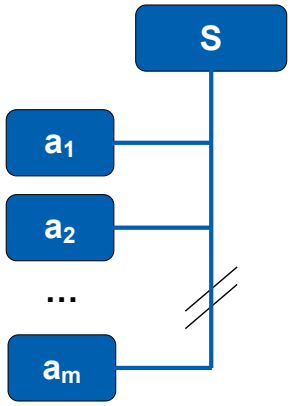
Comutatoare Neierarhice

Comutatoare Neierarhice

- Asigura conexiunea intre elemente de acelasi tip (e.g. P-P; M-M)
- Tipuri:
  - Comutatoare Duplex
  - Legatura multipla – tip central
  - Trunchi K neierarhic
- Structurile cu comutatoare asigura infrastructura de comunicatie

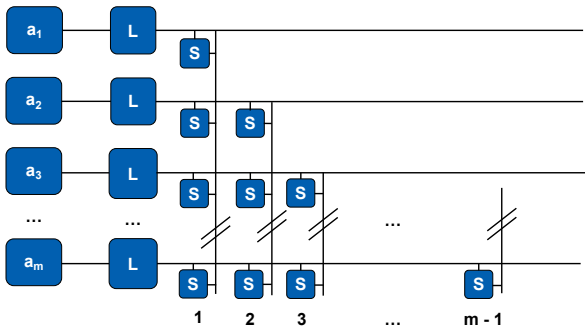
Comutatoare Neierarhice Duplex

- Comutatoare Duplex Neierarhice
  - S{Dn; ma, c:1; mSp}
  - Bus Inlantuit



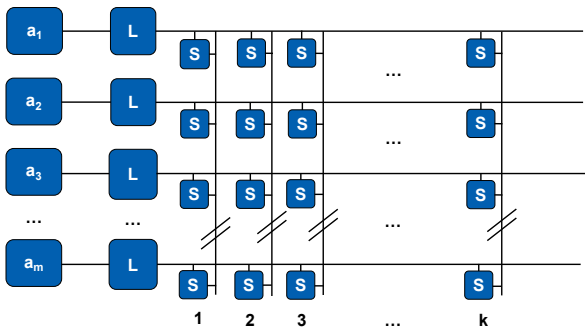
Comutatoare Neierarhice Legatura Multipla

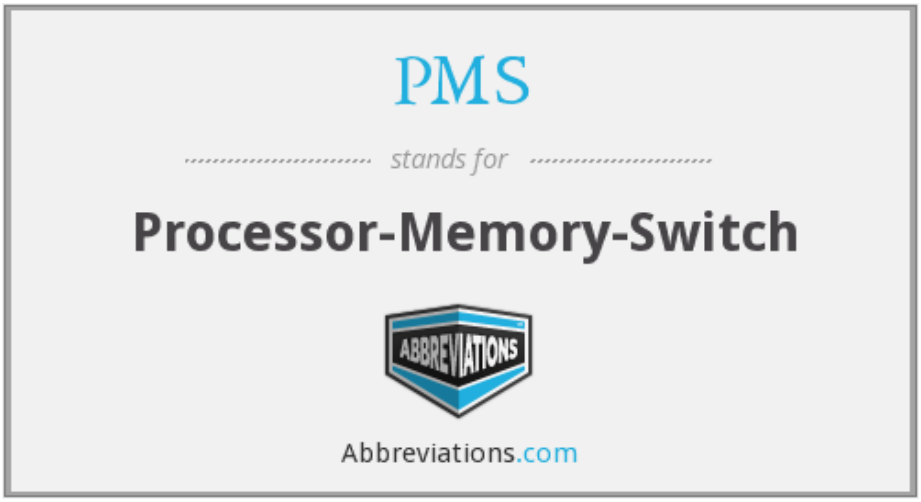
- Legatura multipla – tip central
  - S{lm-neierarhica; ma, c:m/2; m(m-1)/2 Sp}



Comutatoare Neierarhice Trunchi-K

- Trunchi-K neierarhic
  - S{tk; ma, c:min(k,m/2); m*k Sp}

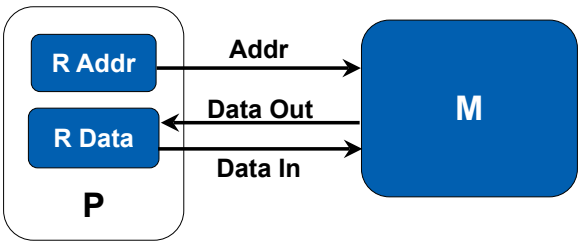




Exemple de Comutatoare Procesor Memorie

Comutatoare Procesor Memorie

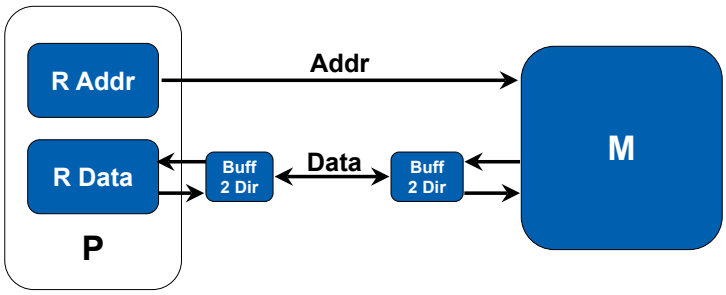
- Switch Processor Memory → S_{PM}
- Liniile de adrese si date sunt complet separate



R/W – Disjuncte → No Switch!
Nu se justifica ambele linii

Comutatoare Procesor Memorie

- Switch Processor Memory → S_{PM}
- D_{In} si D_{Out} sunt multiplexate prin magistrala de date

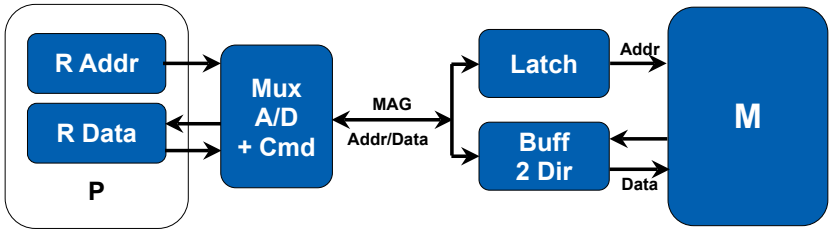


Bufferele sunt fie MUX fie TS



Comutatoare Procesor Memorie

- Switch Processor Memory → S_{PM}
- Addr, D_{In} si D_{Out} sunt multiplexate prin aceeasi magistrala



Bufferele sunt fie MUX fie TS

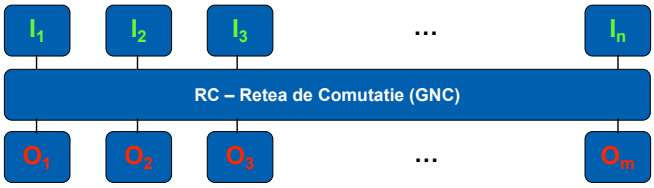




Interconectarea in Sistemele cu Resurse Multiple

Interconectarea in Sistemele cu Resurse Multiple

- In structurile SIMD si MIMD, retelele de comutare (de interconectare a resurselor), au rolul cel mai important in asigurarea performantelor sistemului!
- Realizarea interconectarii resurselor (P, M & I/O) se face printr-o retea de comutatie (RC)



- I_i = Intrari & O_j = Iesiri; RC conecteaza n·m resurse
- Net (n, m) = Generalised Connection Network – graf in care arcele conecteaza perechi de resurse

Retele de Comutatie

- O astfel de RC trebuie sa respecte niste reguli:
  - O intrare poate fi conectata la orice iesire
  - O intrare poate fi conectata simultan la mai multe iesiri
    - Astfel vom avea Broadcast partial sau total
  - O iesire poate fi conectata numai la o intrare
  - Concurenta maxima e data de min (m, n)
- Din aceste reguli rezulta ca RC ar trebui sa realizeze n·m corespondente == GCN!



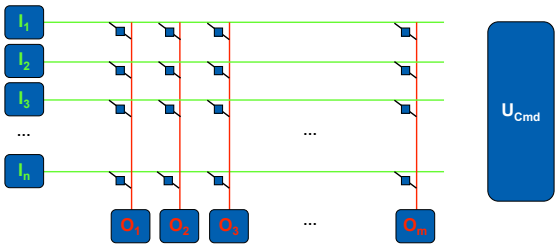
GCN – Generalised Connection Network

- Un GCN poate fi implementat
  - Printr-o structura origanizata pe un singur nivel in care se stabilesc legaturi directe intre intrari si iesiri (crossbar)
  - Prin utilizarea unor subretele de dimensiuni (mai) reduse conectate ierarhic pe mai multe niveluri
- Caracteristici pentru GCN sunt doi parametrii
  - Intarzierea de propagare (Δt) – timpul de raspuns ce depinde de numarul maxim de comutatoare elementare (de tip poarta) ce asigura legatura intre intrare si iesire
  - Complexitatea (nC) – numarul total de comutatoare elementare utilizate in implementarea GCN-ului



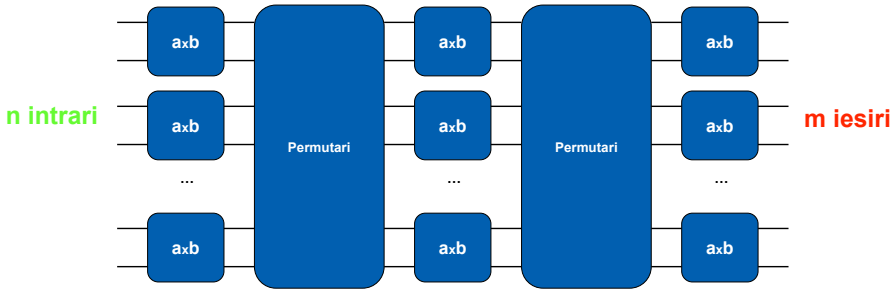
Implementari ale RC (GCN)

- Comutare directa – Cross Bar cu n·m comutatoare (Δt = 1)
- Unitatea de comanda trebuie sa rejecteze 2 I (intrari) / 1 O (iesire)
- O implementare cu comutare directa
  - Este greu de proiectat fizic (nC = n·m)
  - Nu se preteaza la VLSI & este inflexibila la dezvoltare ulterioara
- U_{Cmd} controleaza toate switch-urile elementare



Implementari ale RC (GCN)

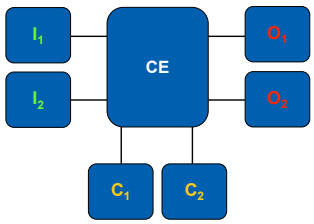
- Prin realizarea unor subretele de comutare organizate ierarhic, pe mai multe niveluri
- Aceasta implementare necesita k ≈ $\log_2 n$ comutatoare elementare → reduce complexitatea (nC) & creste timpul de comutare (Δt)
- Comutatoarele elementare sunt de tip Cross Bar (axb), unde a « n si b « m
- Intre niveluri se aseaza permutarile de interconectare



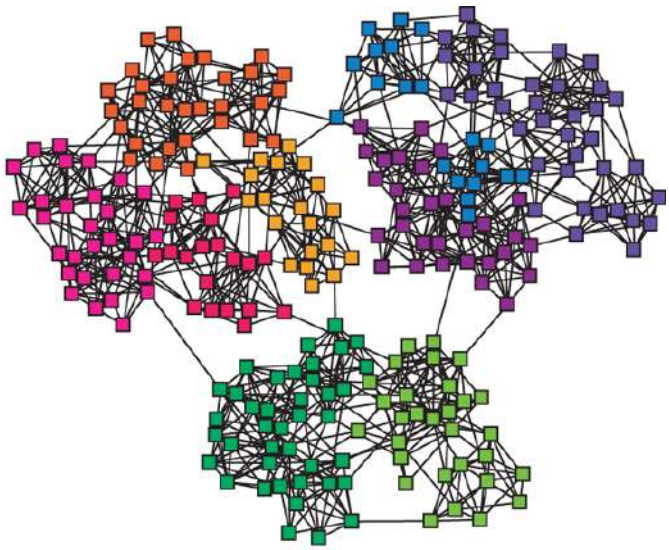
Comutatoare Elementare

- Un comutator elementar are 2 intrari, 2 iesiri si 2 biti de comanda:

C_1	C_2	Connection	
0	0	$O_1 = I_1; O_2 = I_2$	Direct
0	1	$O_1 = I_2; O_2 = I_1$	Inversat
1	0	$O_1 = I_1; O_2 = I_1$	Broadcast Superior
1	1	$O_1 = I_2; O_2 = I_2$	Broadcast Inferior



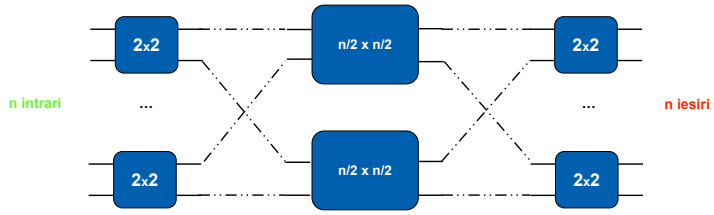
- Aceste module se pot integra pe baza permutarilor elementare
- In acelasi mod se pot construi si comutatoare elementare 4x4, 8x8, etc.



Retele de Comutare Ierarhice

Retele de Comutare Ierarhice

- Realizate din CrossBar-uri de dimensiuni mici asezate pe mai multe niveluri de interconectare:
  - Creste timpul de intarziere datorita utilizarii mai multor module – dar se mentine ∀intrare ↔ ∀iesire
- Exemplu – structura Beizer (Benes):
  - Retea nxn realizata cu doua module n/2 x n/2 (se pot imparti la randul lor in subretele mai mici) si 4n module de tip 2x2
  - Complexitatea este de (4n log n – 2n)

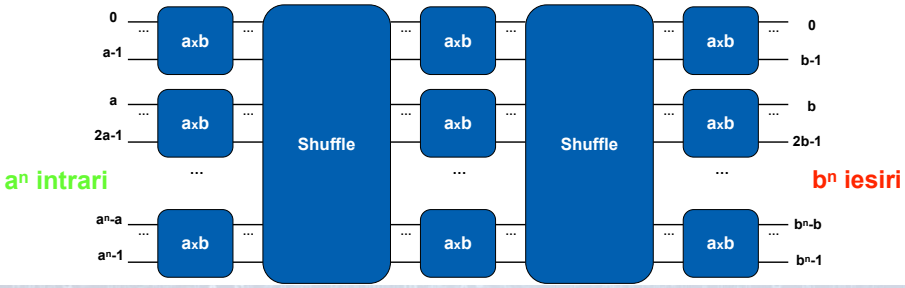




Retele de Comutare de tip Delta

Retele de Comutare de tip Delta

- Sunt comutatoare ierarhice cu a^n intrari si b^n iesiri ce utilizeaza CrossBar-uri (CB) axb si retele de permutare de tip intercalare perfecta (Shuffle)
- Reteaua Delta are n niveluri ierarhice:
  - Nivelul 0 contine a^{n-1} CB-uri de tip axb, pentru a conecta an intrari cu a^{n-1}xb iesiri
  - Nivelul 1 contine asadar a^{n-1}xb intrari conducand la a^{n-2}xb module CB

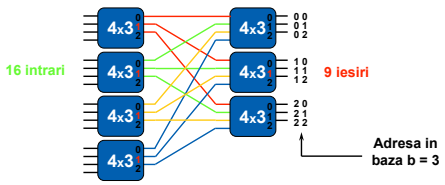


Retele de Comutare de tip Delta

- Astfel, in general:
  - Nivelul i contine $a^{n-i}b^{i-1}$ module CB de tip axb
- Intreaga retea contine:
  - $(a^n-b^n)/(a-b)$ CB-uri de tip axb cand $a \neq b$
  - nb^{n-1} CB-uri de tip axb cand $a = b$
- Destinatia este $D = (d_{n-1}\ d_{n-2}\ \ldots\ d_1\ d_0)$ cu $0 \leq d_i < b$
- Cifrele de reprezentare in baza b, a adresei destinatiei vor controla modulul CB de pe nivelul respectiv (d_i controleaza CB-ul i)
- Functia Shuffle-ului este de a interconecta nivelurile retelei Delta

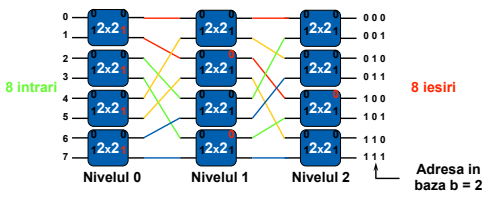
Exemplu – Delta 16x9

- Presupunem ca avem 16 procesoare si 9 memorii
- 16 = 4^2 si 9 = 3^2 → folosim 2 niveluri de CB 4x3
- Daca vrem acces la memoria 4 (11 in baza 3) atunci se merge pe 1 (niv 0) si 1 (niv 1)!



Exemplu – Delta 8x8

- Presupunem ca avem 8 procesoare si 8 memorii
- 8 = 2^3 → folosim 3 niveluri de CB 2x2
- Daca vrem acces la memoria 4 (100 in baza 2) atunci se merge pe 1 (niv 0), 0 (niv 1) si 0 (niv 2)!





Retele Bazate pe Rutare

Retele Bazate pe Rutare

- Au ca nucleu legaturi directe intre o resursa si cele considerate vecine
- Resursele pot fi distincte dar toate au in comun un router ce se ocupa de comunicarea prin mesaje
- Traditional aceste retele sunt modelate prin grafuri cu urmatoarele proprietati de baza:
  - Gradul nodului (al resursei) – defineste numarul de canale de conexiune ale resursei la vecinii sai
  - Diametrul – distanta maxima intre doua resurse ale retelei
  - Regularitatea – nodurile/resursele care au/nu au acest grad
  - Simetria – o retea este simetrica daca “arata la fel” pentru orice nod
- Obs: acest tip de retea nu foloseste comutarea de circuite (CB), ci rutarea – legatura intre noduri/resurse e permisa doar cand este ceruta & permisa

Retele Bazate pe Rutare – Caracteristici

- Topologia – defineste modul in care nodurile sunt interconectate
- Rutarea – stabileste calea de date pe care un mesaj o urmeaza de la sursa la destinatie
- Comutarea – mecanismul ce determina cum si cand un canal de intrare este conectat la unul de iesire
- Obs: trebuie avute in vedere:
  - Alocarea de buffere
  - Controlul fluxului de date



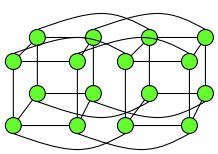
Retele Bazate pe Rutare – Topologii

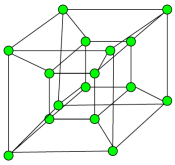
- Topologii ortogonale:
  - Nodurile sunt aranjate intr-un spatiu n-dimensional ortogonal
  - Fiecare legatura produce o deplasare intr-o singura dimensiune
- Topologii strict ortogonale (mai interesante):
  - Fiecare nod are cel putin o legatura in fiecare dimensiune
  - Trecerea intr-o noua dimensiune se poate face din orice nod
  - Rutarea e mai simpla si implementarea HW e mai eficienta
  - Nodurile pot fi numerotate folosind coordonatele lor in spatiul n-dimensional
  - Procesul de rutare se face pe baza diferentei intre coordonate
  - Obs: Distanta dintre noduri poate fi calculata direct folosind adresele nodurilor respective



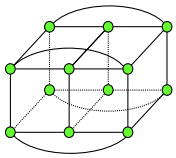
Retele Bazate pe Rutare – Topologii

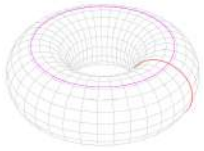
- Hipercub:
  - 2 cuburi unul intr-altul
  - Din orice nod se poate ajunge pe orice nivel
  - Se poate optimiza drumul

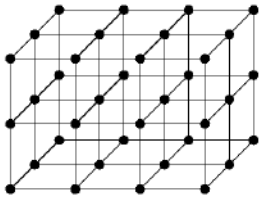




- Tor:



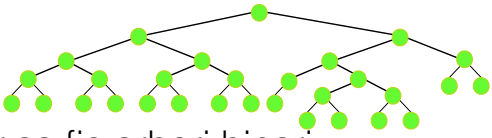




- Plasa n-dimensionala:

Retele de Tip Arbore

- Sunt un alt tip de retele de comutare



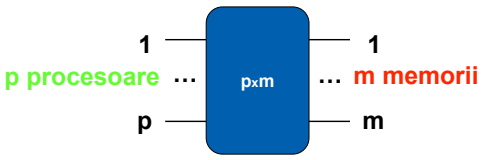
- Nu e neaparat necesar sa fie arbori binari
- Fiecare nod al arborelui poate avea la randul sau un subarbore – subarbori multiplii
- Similar – la structurile cu hipercub, fiecare nod poate fi inlocuit cu un nou cub!
- Exista mai multe cai posibile pentru mesaje – totdeauna se cauta drumul de lungime minima



Performantele Retelelor de Comutare

Performantele Retelelor de Comutare

- Analizam performantele retelelor ce asigura comutarea intre p procesoare si m memorii



- Analiza va fi realizata pentru retele de tip CB si Delta
- Rata de servire – media cererilor de acces la memorii acceptate intr-un ciclu (toate memoriile)
- Ciclu de servire – timpul necesar pentru ca o cerere de acces la memorie sa se propage prin logica de comutare + timpul efectiv de acces la memorie (propagarea inversa)

Presupuneri / Restrictii

- Nu se fac deosebiri intre ciclurile de citire si de scriere
- Fiecare procesor va genera cereri independente pentru acces la un modul de memorie
- Cererile sunt uniform distribuite in cadrul tuturor modulelor de memorie – fiecare procesor cu un modul de memorie
- La inceputul fiecarui ciclu
  - Fiecare procesor genereaza o noua cerere de acces cu probabilitatea r (media cererilor generate de fiecare procesor intr-un ciclu)
  - Nu exista coada de asteptare → dialog direct
- Cererile neacceptate intr-un ciclu sunt ignorate
  - Cererile din ciclul urmator sunt independente de cele din ciclul anterior
  - Cererile neacceptate trebuiesc relansate intr-un ciclu urmator



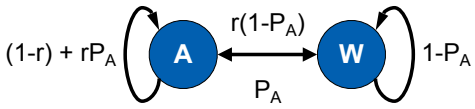
Comportarea unui Procesor intr-un CB

- Se va utiliza un graf Markov
- Un procesor are 2 stari posibile:
  - A = starea activa – cererea este acceptata
  - W = starea de asteptare (wait) – cererea este respinsa
- q_A si q_W sunt probabilitatile ca procesorul sa se afle in starile A si W respectiv
- P_A = probabilitatea ca cererea sa fie acceptata – adica de a trece din starea W in starea A
- Astfel, putem defini:

$$q_A = \frac{P_A}{P_A + r(1 - P_A)} \qquad q_W = 1 - q_A$$



Comportarea unui Procesor intr-un CB



- Se considera rata de cereri r ca fiind statica
  - In realitate r e diferita luandu-se in considerare conflictele de acces
- Modulele de memorie primesc r' > r cereri de acces:

$$r' = rq_A + q_W = \frac{r}{r + (1-r)P_A} \qquad P_A = \frac{m}{r'p}\left(1 - \left(1 - \frac{r'}{m}\right)^p\right)$$

Analiza Retelei de Tip CrossBar

- Relatiile dinainte, definesc r' ca o functie de PA si PA ca o functie de r' → definesc un proces iterativ:
  - Initiem rata de transfer r' = r
  - P_A poate fi considerat ca o masura a cererilor de acces nerezolvate
  - Daca P_A este mare atunci sunt putine stari de asteptare
- Fie $\overline{w}$ media numarului de cicli de asteptare pentru o cerere
  - O cerere rejectata de i ori va astepta i ciclii $\overline{w} = \sum_{i=1}^{p} i(1-P_A)^i = \frac{1-P_A}{P_A}$
- In practica, cererile ce produc conflicte de acces la memorii, nu sunt rejectate, ci introduse intr-o coada de asteptare si servite FIFO

Analiza Retelei de Tip Delta

- Consideram o retea Delta ($a^n x b^n$) formata din CB-uri elementare (axb)
  - a^n procesoare si b^n memorii
- Fiecare nivel al retelei Delta e controlat de unul din bitii de reprezentare a adresei destinatie, in baza b!
- Orice modul CB axb e independent de celelalte module → putem considera:
  - Cererile independente si uniform distribuite
- Daca r este rata de cereri la intrarea in axb atunci: $B(a,b) = b - b\left(1 - \frac{r}{b}\right)^a$
- Impartim la b → rata de cereri la fiecare iesire din CB-ul curent: $\frac{B(a,b)}{b} = 1 - \left(1 - \frac{r}{b}\right)^a$



Analiza Retelei de Tip Delta

- Pentru fiecare nivel intermediar al retelei Delta:
  - r_{out} (rata de cereri pentru fiecare iesire) este o functie de r_{in} (rata de cereri de la intrare)

$$r_{out} = 1 - \left(1 - \frac{r_{in}}{b}\right)^a$$

- Rata de cereri de la iesirea unui nivel r_n este rata de cereri de intrare pentru nivelul adiacent:
  - Rata de cereri de la ultimul nivel determina rata de transfer a retelei de tip Delta
- Fie r_i rata de cereri de la iesirea unui nivel in reteaua Delta → rata de cereri a unei RC Delta, generata de fiecare procesor este:

$$B(a^n, b^n) = b^n \cdot r_n \quad \text{unde} \quad r_i = 1 - \left(1 - \frac{r_{i-1}}{b}\right)^a \quad \text{si} \quad r_0 = r$$

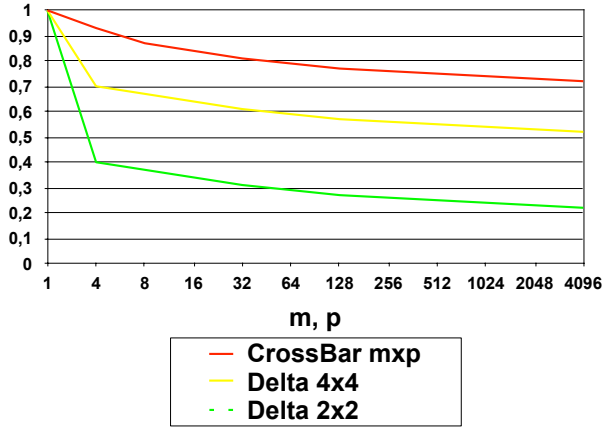


Analiza Comparativa Delta / CB

- Probabilitatea ca o cerere sa fie acceptata este:

$$P_A = \frac{b^n \cdot r_n}{a^n \cdot r}$$

- Ce influenta au m si p-ul unei retele?
  - CrossBar **m**x**p** cu r=1 rata de cereri a fiecarui procesor
  - Delta 4^nx4^n cu 4x4
  - Delta 2^nx2^n cu 2x2
- Pentru m mare, P_A-ul scade puternic
  - Multe rejectii si conflicte
  - Limita la 16^nx16^n este 0.6



Ce am invatat in aces curs?

- Comutatoare Ierarhice
- Comutatoare Neierarhice
- Exemple de Comutatoare Procesor Memorie
- Interconectarea in Sistemele cu Resurse Multiple
- Retele de Comutare Ierarhice
- Retele de Comutare de tip Delta
- Retele Bazate pe Rutare
- Performantele Retelelor de Comutare

Thank you for your attention.





emil.slusanschi@cs.pub.ro
cs.pub.ro